



Automatic tuning of hyperparameters using Bayesian optimization

A. Helen Victoria¹ · G. Maragatham¹

Received: 21 December 2019 / Accepted: 15 May 2020 / Published online: 25 May 2020
© Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract

Deep learning is a field in artificial intelligence that works well in computer vision, natural language processing and audio recognition. Deep neural network architectures has number of layers to conceive the features well, by itself. The hyperparameter tuning plays a major role in every dataset which has major effect in the performance of the training model. Due to the large dimensionality of data it is impossible to tune the parameters by human expertise. In this paper, we have used the CIFAR-10 Dataset and applied the Bayesian hyperparameter optimization algorithm to enhance the performance of the model. Bayesian optimization can be used for any noisy black box function for hyperparameter tuning. In this work Bayesian optimization clearly obtains optimized values for all hyperparameters which saves time and improves performance. The results also show that the error has been reduced in graphical processing unit than in CPU by 6.2% in the validation. Achieving global optimization in the trained model helps transfer learning across domains as well.

Keywords Hyperparameters · Optimization · CIFAR-10 · Black box function

1 Introduction

Due to the immense growth of data from various sources, digital data can be used for more exciting applications. Deep learning has the scope to use the deluge of data to build sophisticated and intricate deep neural network models. Such customization engenders a large number of layers by increasing the number of hyperparameters. Since deep learning is considered to be a black box approach the researcher doesn't have much scope in hand tuning the parameters as the layers are hidden and there are many hyperparameters related to network structure and training algorithms as well. Bayesian optimization is most useful while optimizing the hyperparameters of a deep neural network, where evaluating the accuracy of the model can take few days for training. The aim of optimizing the hyperparameters is to find an algorithm that returns best and accurate performance obtained on a validation set. The optimizer finds the best

hyperparameters which yield the best score on the test set. Bayesian optimization is also widely used in diverse design problems in different fields, such as gait recognition (Feurer et al. 2019), environmental sustainment and monitoring (Marchant and Ramos 2012), algorithm configuration using optimization (Hutter et al. 2014), automatic machine learning (Snoek et al. 2012; Bergstra et al. 2011; Hoffman et al. 2014), reinforcement learning (Brochu et al. 2009), and big data applications (Shahriari et al. 2016). Bayesian optimization works fairly well when the dataset for classification is non-linear, complex and noisy, since the computation for identifying the hyperparameters is expensive thereby affecting model performance. The main motivation of this work is to reduce the training time of deep neural network using Bayesian optimization without sparing the model performance.

2 Related work

2.1 Multiple parameter optimization

Deep learning architectures has various layers hence before fitting into a model we have to configure all the hyperparameters that leads to proper learning of the model which can yield better classification. The choice of hyperparameters

✉ A. Helen Victoria
helenvia@srmist.edu.in

G. Maragatham
maragatg@srmist.edu.in

¹ Department of Information Technology, SRM Institute of Science and Technology, Kattankulathur, Chennai, Tamil Nadu, India

plays a major role in bettering the predictive performance. The aim of optimizing hyperparameters is to choose the hyperparameter values that return the best results in the validation.

The hyperparameter optimization is represented by the Eq. 1.

$$y^* = \operatorname{argmin} f(y). \quad (1)$$

y belongs to domain space Y , $f(y)$ represents the objective score to minimize the error rate evaluated against the validation set, whereas y is said to be the set of hyperparameters which is a value in the domain where the evaluation of hyperparameter optimization is more expensive. With complex deep neural network models and a large number of hyperparameters, training time is longer and is possibly intractable by human. The traditional hyperparameters optimization techniques are grid and random search which is little better than manual tuning but they do not consider previous results therefore they cannot choose the next set of hyperparameters while training the model. Moreover, Grid search usually trains all possible combinations of hyperparameters, if we want to optimize 6 parameters using 10 different values for each parameter there will be a need of 10^6 evaluations which will end up in training the network for very long time. This can be handled by random search by choosing parameters randomly, eventually reducing its training time, but with the disadvantage of probably not covering the entire parameter space. Grid and random search (Bergstra and Bengio 2012) are completely unaware of previous evaluations which is a major setback for using these optimization techniques. This could be overcome by Bayesian model-based optimization. Bayesian optimization keeps track of past results to choose the best hyperparameters for evaluation (Dewancker et al. 2015). Bayesian optimization has been used in machine learning models and simulations (Bergstra et al. 2013). Mainly in computer vision-based algorithms, we use feed forward network architectures for tuning hyperparameters for better performance of the model (Calandra et al. 2016). It helps in formulating the tedious task of optimizing many parameters. In addition to the grid and random search optimization techniques, there are others like the random forest as well. Bayesian optimization has been successfully used in robotic applications as well. It has been used to find the good set of gait parameters in many experiments (Kochanski et al. 2017). Bayesian optimization has been used even to complex real-world problems like optimizing for a better dessert (Lorenzo 2017). Bayesian optimization is highly used when the data is complex, nonlinear and noisy. It is used in human gait classification for radar signals to optimize the model (Le et al. 2018). Bayesian optimization has been used to lessen the number of evaluations in harmonic balance simulator (Knudde 2018).

Expected Improvement based Bayesian optimization is also used for automating analog based sizing (Lyu et al. 2018). In this paper, we explore how Bayesian optimization helps in hyperparameter tuning, thereby reducing the time involved and improving performance.

3 Hyperparameters of deep neural networks

Hyperparameters are a set of parameters used for the learning process during training and testing. The deep learning model will have various instances and weights to learn different feature combinations as patterns. Generally, hyperparameters include the learning rate, number of iterations, hidden layers, batch size, activation functions, momentum, and regularization. For convolutional neural networks which is used for image classification, the field size of convolutional layers and pooling layers and also the step size which is handled by the stride parameter is considered. The parameters might be an integer, or a continuous or categorical variable which ranges from the lower to upper bound values. Intermediate layers may generate new hyperparameters, that is, the number of neurons may vary across layers.

Consider the hyperparameters y_1, y_2, y_3 in the configuration space Y , y belongs to Y such that $Y = y_1 * y_2 * y_3$ can be initialized before training the model. During the training procedure, the best hyperparameters are to be chosen in order to efficiently train the model which can have major impact on the model. If the model has less learning rate, important patterns could end up being inadvertently left out. The set of parameters should be in right combination having minimum loss function or maximizing the performance or accuracy of the model. Thereby tuning of hyperparameters can be formulated as an optimization problem, similar to fuzzy optimization (Angelov 1994; Angelov et al. 2011; Baruah and Angelov 2014) that is done in raw data. Hyperparameters of deep learning model remain fixed throughout the training procedure which helps to increase the accuracy of the model, also considering the memory cost and time for training the model eventually reducing the value of loss function.

The hyperparameters chosen vary from model to model, based on the problem statement. There is no such set of best hyperparameters that fits for all models. In this work, we have considered the learning rate, momentum, regularization, and network depth. The learning rate helps identify generic patterns in the image, while momentum helps scan the entire search space without missing key points. Regularization helps the model to generalize better, without overfitting, which helps make better predictions as well.

4 Bayesian optimization

Deep learning optimization comprises black box optimization problems in which the objective function, $f(y)$, is a black box function. It is critical to reduce the number of samples throughout the number of steps across the layers. Bayesian optimization is much useful in this kind of domain where human expertise could not contribute for better accuracy. Bayesian optimization includes prior information about the function f , and updates posterior information, which helps reduce loss and maximize the model's accuracy. Hand tuning is a very hard procedure to reproduce as its based-on trial and error. While grid search is not scalable for higher dimensions, random search, which is the better of the two, is rather more like the greedy approach, settling for local optima and failing to end with global optima. Other evolutionary optimization methods require a great number of training cycles and are noisy as well. Bayesian optimization overcomes all these constraints by efficiently unveiling the global optima of the black box function of the neural network. Its scales well with utmost resource utilization, handling noisy data well exploiting non continuous spaces to attain global minima.

The optimization procedure is based on Baye's Theorem (Kramer et al. 2011) in the equation which states for a model Z and observation Y (Kramer et al. 2011)

$$P(Z|Y) = (P(Y|Z)P(Z))/P(Z)$$

where $P(Z|Y)$ is the posterior probability of Z given Y , $P(Y|Z)$ is the likelihood of Y given Z , $P(Z)$ is the prior probability of Z , and $P(Z)$ is the marginal probability of Z . Bayesian optimization is used to find the minimum of a function, $f(y)$, on a bounded set, Y . In Fig. 1, Bayesian optimization is used to find the best measure among the sample values where the function is said to be minimum. The deep learning model, with extra evaluations for function $f(y)$, is

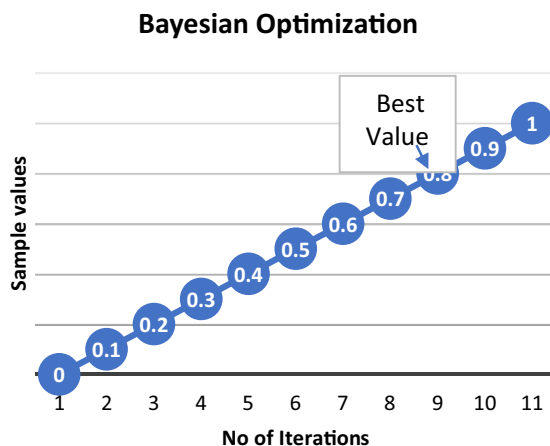


Fig. 1 Bayesian optimization

acceptable to make better decisions. In this paper, an optimal network architecture arrived at by reducing the error in validation is found, using Bayesian optimization.

The major steps in the optimization are as follows:

1. Posterior distribution is adopted using Gaussian process updating the previous results of Function F .

The black box approach is required to remember the entire training data as it handles the non-parametric method. The Gaussian process has certain observed prior values that can be converted to posterior values, directly on the functions. With this context Gaussian process is applied here for classification. When the Gaussian process maps a set of prior points that can be used to predict function points for any new test data, the performance of the model improves.

2. Choosing the best point for the function, F , by using an acquisition function, A (Joy et al. 2016).
3. Identifying the proposed sampling points obtained by the acquisition function.

At the points where the objective function is maximum and the location where the uncertainty of the prediction is high, the expected improvement acquisition function is used. It proceeds its work, based on uncertainty.

$$\text{Expected Improvement} = ((Ex - \text{Optfn at Mx}) - Le) (\text{cdf}). \quad (2)$$

In Eq. 2 above, Ex represents the mean of sample points Optfn at Mx represents the Optimizing function at Maximum sample point which has the limitation for search space represented at Le . cdf represents the cumulative distribution function that considers all the points till that sample point under gaussian distribution.

4. Using an objective function to obtain results in the validation set.
5. Augmenting the best optimized sample points to the data already selected.
6. Updating the statistical Gaussian distribution model.

This is repeated until certain number of iterations to tune the validation set for obtaining optimized parameters that works for better classification.

Figure 2 explains the framework of Bayesian optimization with respect to the deep learning model.

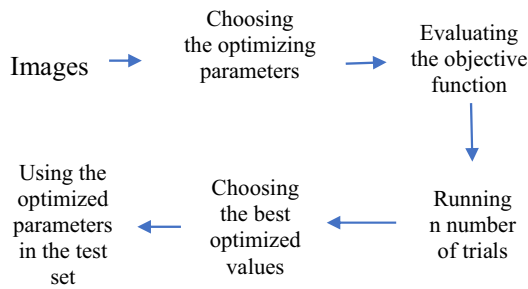


Fig. 2 A framework of Bayesian optimization

5 CNN network architecture

Convolutional neural networks have produced excellent results in fields like image classification (Le Cun et al. 2015), speech recognition (Guiming et al. 2016), and object identification (Erhan et al. 2014). The CNN is very successful because it learns the features by itself which yields good classification even without detailed domain knowledge.

This architecture explained in Fig. 3 has huge dataset of images followed with repeated layers of convolutional layer, batch normalization layer and a rectified linear unit (ReLU) layer for each iteration followed by flattened fully connected layer which is given to SoftMax classifier in the output layer which gives the trained output labels. Each has a stack of blocks. In this paper, we use a max pooling layer which helps retain the maximum pixel values, along with its spatial arrangement, which reduces the number of model parameters used during training. Each set of layers helps to extract features through various iterations. These layers are followed by a fully connected layer with an exact number of neurons, as per class labels and the SoftMax activation layer, to choose the exact match with the test data. Feature Map is given by F_m

$$F_m = b + \sum k_j * I_i,$$

b represents bias, and k represents the kernel for j th iteration, and $*$ represents convolution operation of Kernel and

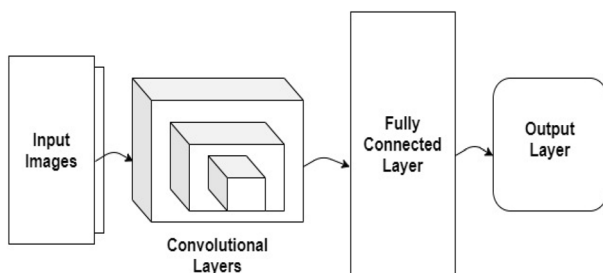


Fig. 3 Convolutional neural network architecture

the Input across iterations. The convolution operation helps retain generic features and the deeper network gets customized to the problem statement, obtaining more relevant and complex features.

6 Experimental setup and analysis

The CNN Model given in this paper is used to predict the correct class of CIFAR-10 dataset which has 60,000 32×32 images with 10 class labels which has 50,000 images for validation and 10,000 images as test images. The hyperparameters chosen for Bayesian optimization are given in Table 1.

The total number of convolutional layers is 3*stacks. Each stack has a convolutional layer, batch normalization layer, and ReLU layer. The image size is $32 \times 32 \times 3$, where 3 represents the depth of the image. The mini batch size is 128. The initial number of filters is $IF = \text{round}(16/\sqrt{Ov * \text{Network Depth}})$, where Ov represents the optimizing variables used to maintain the same number of trainable parameters. The filter size across layers is 32×32 , 16×16 , and 8×8 respectively. The key parameter is the learning parameter that reduces training time by minimizing the local minima. An increase or decrease in the learning rate depends on the gradient loss function error, and it is usual to start with a small learning rate. The equation for learning parameter α is given by Eq. 3.

$$\theta_{new} = \theta_{old} - \alpha (\delta / \delta \theta_{old}) * \text{gradient}. \quad (3)$$

Network depth helps to identify the unique feature of the images by fine tuning the network depth. The next parameter chosen is momentum, which updates previous gradients faster than the normal gradient descent. Adding momentum to the gradient descent helps alleviate noisy calculations and yields better estimates. Momentum is given by Eqs. 4 and 5,

$$V_t = \delta V_t - 1 + \alpha \Delta w L(W, Z, y), \quad (4)$$

$$W = W - V_t, \quad (5)$$

where δ is the momentum, Δw is the gradient change, α is the learning parameter, V_t represents change in momentum represents the weight. Momentum with the stochastic

Table 1 The chosen hyperparameters to be optimized

Hyperparameters	Initial value	Final value	Type
Network depth	1	3	int
Learning rate	0.06	0.01	log
Momentum	0.8	0.95	log
Regularization	0.002	2.05e−09	log

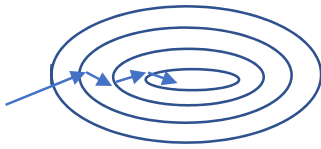


Fig. 4 Stochastic gradient descent with momentum

gradient takes the exponentially weighted averages of the gradients. δ is the momentum which takes the average of the previous values and current values ranging from 0 to 1. The main advantage of using momentum is to smooth out the gradient descent steps which minimizes vertical oscillations forwarding a better path to local optima with lesser number of iterations than stochastic gradient.

Figure 4 depicts how the momentum helps in accelerating the right direction towards the optimum.

Tuning of hyperparameters is done in the validation set hence there is chance for the data to overfit in the test data due to generalization error, this could be overcome by augmenting the data by reflection, rotation and translation and cropping. Augmentation helps reduce overfitting only to an extent. The other parameter is regularization which helps in reducing the generalization error in this dataset. Regularization helps to reduce the size of search space by reducing the weight by small factor called weight decay. The aim of tuning the regularization is to learn patterns properly by ignoring noise, and reducing the complexity of the system.

Optimization process that requires a loss function to identify model error. Mean squared error is the loss function used when training the model. Regularization term (λ) should have an optimum value since if its zero there will be no impact in the loss function, if its high it might eliminate the presence of a neuron in the layer which might lead to underfitting. In this paper, we have used ridge regularization, which is the sum of the squares of all the feature weights. The loss function, L , is given by Eq. 6,

$$L(y, z) = \sum_{i=1}^n (z_i - y_i)^2 + \lambda \sum_{i=1}^n \theta_i^2. \quad (6)$$

where θ_i represents the feature vector, and y, z are the input parameters for the i th iteration. The feature vector represents the intermediate learning images obtained from each iteration after using the loss function. The feature vector is automatically identified in deep learning only with the help of input images, after proper training at each level. The loss function is used to train the model better in order to have better accuracy at each and every iteration. Ridge regularization offers better accuracy when the output variable is the function of all the input variables. Tuning regularization and the learning rate must go hand in hand.

Table 2 Hyperparameter values during tuning

I	Initial learning rate	Momentum	Regularization	Network depth
1	0.067	0.858	0.0002	1
2	0.01	0.838	0.002	2
3	0.032	0.936	2.819e−07	2
4	0.001	0.844	1.784e−09	3
5	0.010	0.919	2.055e−09	2

Table 3 Best observed feasible points

Network depth	Learning rate	Momentum	Regularization
2	0.003	0.936	2.819e−07

Table 4 Validation and test error values in the GPU & CPU

Validation error	Test error	Test error 95 CI	Processor
0.212	0.216	0.208	GPU
0.225	0.230	0.222	CPU

7 Experimental results

The optimal hyperparameters obtained using Bayesian optimization are given in Table 2. The values given below are executed in the graphical processing unit (GPU) over iterations. Optimization happens in each and every layer to obtain maximum optimization. In this paper we have done only 5 iterations(I) which we can extend up to 15–20 iterations to improve optimization, even if we extend the number of iterations Bayesian optimization automatically stops once it reaches the maximum optimized values.

Table 3 gives the best feasible values for the hyperparameters obtained from the validation set which is used as parameters for the test set, which fairly reduces the generalization error and helps in increasing accuracy.

Table 4 indicates the error values in CPU and GPU. From the observed values it is clearly shown that error has been reduced in GPU than in CPU which is 6.2% more with respect to GPU in validation error.

Figure 5 gives the results of the validation accuracy of the sample images in the CIFAR 10 Dataset. The optimized parameters obtained from the validation set are tested against the test data to check the model's performance. The test error rate in simple Net image classification using the CIFAR 10 dataset is 0.26 (Mantovani et al. 2015), which is little when compared to our test error rate using Bayesian optimization.

The deep convolutional neural network function evaluation is very expensive and time-consuming. The

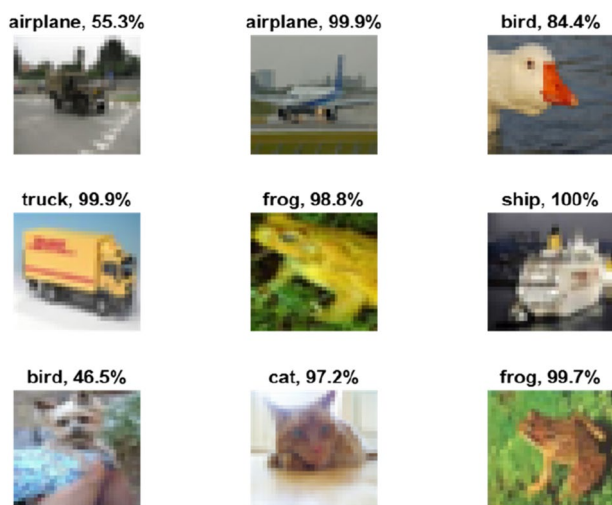


Fig. 5 Validation accuracy of the sample images

acquisition function helps find the search space quickly, so the expensive function is performed only in a specific region space. The objective function, mapped as a Gaussian process model trains the model by means of the objective function evaluation. Gaussian Function is used to track analytically since it induces a posterior distribution over the objective function. The function evaluation is illustrated by a graph in Figs. 6 and 7, which explains how the minimum objective is obtained through the iterations. In every iteration, the samples obtained from the acquisition function are evaluated by the objective function, and the samples added to the data to update its posterior according to Bayes' theorem.

Thus, the hyperparameters are optimized throughout the layers and global optimization is executed across the validation set, thereby reducing the time taken and improving

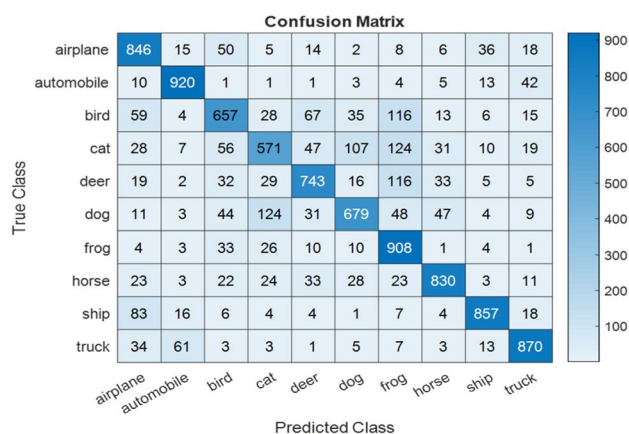


Fig. 6 The confusion matrix for 10 predicted labels across the true class with 10,000 images as test images

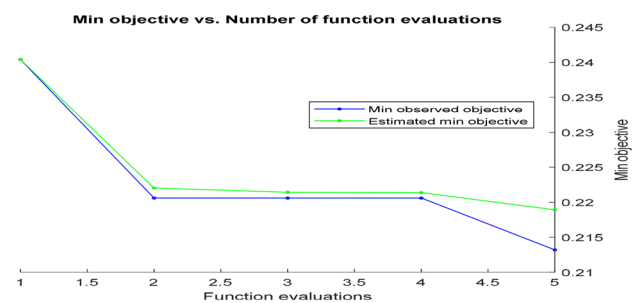


Fig. 7 Minimum objective Vs number of function evaluations

the performance of the model. This shows that there exists a trade-off between the model training and hyperparameter tuning time, which is quite useful (Hasanpour et al. 2018) when the trained model is used for other domains as well.

8 Conclusion and future work

Bayesian optimization is most useful in times when the function is expensive, not easily differentiable and expressive. In this paper we have seen how to use Bayesian optimization for black box functions which doesn't reveal how the hyperparameter and features are extracted during training. In most of the domains search space is huge for example, identifying the effective candidate drug from the chemical search space has number of samples which needs the black box function to be optimized. Hyperparameter tuning certainly improves validation errors. The limitation in Bayesian optimization is that the acquisition function sets the search space early so at times the model might miss an important feature. Domains wherever function evaluation is expensive Bayesian optimization plays a major role to achieve global optimum. Hyperparameter optimization can also be used in transfer learning approaches, spanning across domains.

References

- Angelov P (1994) A generalized approach to fuzzy optimization. *Int J Intell Syst* 9(3):261–268. <https://doi.org/10.1002/int.4550090302>
- Angelov P, Sadeghi-Tehran P, Ramezani R (2011) An approach to automatic real-time novelty detection, object identification, and tracking in video streams based on recursive density estimation and evolving Takagi-Sugeno fuzzy systems. *Spec Issue Intell Data Process Methodol Prob Realiz Trends* 26(3):189–205. <https://doi.org/10.1002/int.20462>
- Baruah RD, Angelov P (2014) DEC: dynamically evolving clustering and its application to structure identification of evolving fuzzy models. *IEEE Trans Cybernet* 44(9):1619–1631. <https://doi.org/10.1109/TCYB.2013.2291234>

- Bergstra J, Bengio Y (2012) Random search for hyper-parameter optimization. *J Mach Learn Res* 13:281–305. <http://www.jmlr.org/papers/v13/bergstra12a.html>
- Bergstra J, Bardenet R, Bengio Y, Kegl B (2011) Algorithms for hyper-parameter optimization. *Adv Neural Inf Process Syst*. <https://doi.org/10.5555/2986459.2986743>
- Bergstra J, Yamins D, Cox DD (2013) Making a science of model search: hyperparameter optimization in hundreds of dimensions for vision architectures. In: *Proceedings of the 30th International Conference on Machine learning*, Atlanta, Georgia, USA. <https://doi.org/10.5555/3042817.3042832>
- Brochu E, Cora VM, de Freitas N (2009) A tutorial on Bayesian optimization of expensive cost functions, with application to active user modelling and hierarchical reinforcement learning. Technical Report UBC TR-2009-23, Dept. of Computer Science, University of British Columbia. <https://arxiv.org/abs/1012.2599v1>
- Calandra R, Seyfarth A, Peters J, Deisenroth MP (2016) Bayesian optimization for learning gaits under uncertainty an experimental comparison on a dynamic bipedal walker. *Ann Math Artif Intell* 76(1–2):5–23. <https://doi.org/10.1007/s10472-015-9463-9>
- Dewancker I, McCourt M, Clark S (2015) Bayesian optimization primer. https://app.sigopt.com/static/pdf/SigOpt_Bayesian_Optimization_Primer.pdf
- Erhan D, Szegegy C, Toshev A, Anguelov D (2014) Scalable object detection using deep neural networks. In: *Proc. IEEE Conf. on computer vision and pattern recognition*, pp 2147–2154. <https://arxiv.org/abs/1312.2249v1>
- Feurer M, Klein A, Eggenberger K, Springenberg JT, Blum M, Hutter F (2019) Auto-sklearn: efficient and robust automated machine learning, part of the springer series on challenges in machine learning book series (SSCML). https://doi.org/10.1007/978-3-030-05318-5_6
- Guiming D, Xia W, Guangyan W, Yan Z, Dan L (2016) Speech recognition based on convolutional neural networks. In: *IEEE international conference on signal and image processing (ICSIP)*, Beijing, pp 708–711. <https://doi.org/10.1109/SIPROCESS.2016.7888355>
- Hasanpour SH, Rouhani M, Fayyaz M, Sabokrou M (2018) Let's keep it simple: using simple architectures to outperform deeper and more complex architectures. <https://arxiv.org/abs/1608.06037v7>
- Hoffman M, Shahriari B, de Freitas N (2014) On correlation and budget constraints in model-based bandit optimization with application to automatic machine learning. *AI Stat*. <https://proceeding.s.mlr.press/v33/hoffman14.pdf>
- Hutter F, Hoos HH, Leyton-Brown K (2014) Sequential model-based optimization for general algorithm configuration. *LION*. https://doi.org/10.1007/978-3-642-25566-3_40
- Joy TT, Rana S, Gupta S, Venkatesh S (2016) Hyperparameter tuning for big data using Bayesian optimisation. *Proc IEEE Int Conf Pattern Recogn*. <https://doi.org/10.1109/ICPR.2016.7900023>
- Knudde N et al (2018) Data-efficient Bayesian optimization with constraints for power amplifier design. In: *2018 IEEE MTT-S international conference on numerical electromagnetic and multiphysics modelling and optimization (NEMO)*, Reykjavik, pp. 1–3. <https://doi.org/10.1109/nemo.2018.8503107>
- Kochanski G, Golovin D, Karro J, Solnik B, Moitra S, Sculley D (2017) Bayesian optimization for a better dessert. In: *31st conference on neural information processing systems (NIPS)* Long Beach, CA, USA. https://www.researchgate.net/publication/324273062_Bayesian_Optimization_for_a_Better_Dessert
- Kramer O, Ciaurri DE, Koziel S (2011) Derivative-free optimization. In: *Computational optimization, methods and algorithms*. Springer, pp. 61–83. https://doi.org/10.1007/978-3-642-20859-1_4
- Le Cun Y, Yoshua B, Hinton G (2015) Deep learning. *Nature* 521(7553):436–444. <https://doi.org/10.1038/nature14539>
- Le HT, Phung SL, Bouzerdoum A, Tivive FHC (2018) Human motion classification with micro-doppler radar and Bayesian-optimized convolutional neural networks. In: *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, Calgary, AB, 2018, pp 2961–2965. <https://doi.org/10.1109/icassp.2018.8461847>
- Lorenzo PR et al (2017) Hyper-parameter selection in deep neural networks using parallel particle swarm optimization. In: *Proceedings of the genetic and evolutionary computation conference companion*. ACM <https://doi.org/10.1145/3067695.3084211>
- Lyu W et al (2018) An efficient Bayesian optimization approach for automated optimization of analog circuits. *IEEE Trans Circuits Syst I Regul Papers* 65(6):1954–1967. <https://doi.org/10.1109/TCSI.2017.2768826>
- Mantovani RG, Rossit ALD, Vanschoent J, Bischl B, Carvalho ACPLF (2015) To tune or not to tune: recommending when to adjust SVM hyper-parameters via meta-learning. In: *IEEE Proceedings of the international joint conference on neural networks*, At Killarney, Ireland. <https://doi.org/10.1109/ijcnn.2015.7280644>
- Marchant R, Ramos F (2012) Bayesian optimisation for intelligent environmental monitoring. *Proceedings of the IEEE/RSJ international conference on intelligent robots and systems*. *IEEE/RSJ Int Conf Intell Robots Syst*. <https://doi.org/10.1109/iros.2012.6385653>
- Shahriari B, Swersky K, Wang Z, Adams RP, de Freitas N (2016) Taking the human out of the loop: a review of Bayesian optimization. *Proc IEEE*. <https://doi.org/10.1109/JPROC.2015.2494218>
- Snoek J, Larochelle H, Adams RP (2012) Practical Bayesian optimization of machine learning algorithms. *Adv Neural Inf Process Syst*. <https://arxiv.org/pdf/1206.2944.pdf>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.